GIT QUEST | 30-DAY GIT LEARNING SERIES

Day 8: .gitignore

Week 2: Undo and Branch | Level: Absolute Beginner

Created by Rutuja Naik

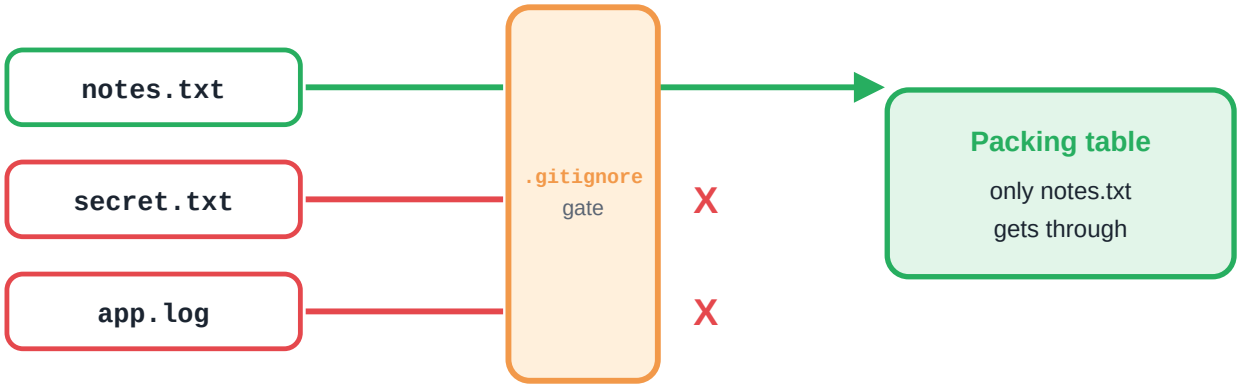
Today's mission

Week 1 is done and Week 2 (Undo and Branch) begins. Some files should never travel with your project: passwords, logs, build output and system junk. Today you learn to keep them out with `.gitignore`.

- Explain what `.gitignore` does and why it matters
- Write patterns: exact names, wildcards and folders
- Check what Git is ignoring and why
- Stop tracking a file that was committed by mistake

1. The big idea: the do-not-pack list

Before a parcel leaves the packing table, someone checks a **do-not-pack list**. `.gitignore` is that list. Git reads it and quietly skips every matching file: it never shows in `git status` and `git add .` leaves it alone.



Kind of file	Examples
Secrets	secret.txt, .env, API keys
Build output and dependencies	build/, dist/, node_modules/
Logs and temp files	*.log, *.tmp
System and editor files	.DS_Store, Thumbs.db, .vscode/

2. Writing patterns

One pattern per line, in a plain text file named `.gitignore` in your project's top folder.

Pattern	Matches
secret.txt	Exactly that file
***.log**	Every file ending in .log
build/	The whole build folder
/todo.txt	Only todo.txt in the top folder
!keep.log	Exception: track this one anyway
# comment	A note to yourself, ignored by Git

A sample .gitignore

```
# Secrets
secret.txt
.env

# Logs
*.log

# Build output
build/
node_modules/

# System junk
.DS_Store
```

3. Step by step

Step 1: Make a lab with some junk

```
$ git init -b main ignore-lab
$ cd ignore-lab
$ echo "My notes" > notes.txt
$ echo "password=1234" > secret.txt
$ echo "debug" > app.log
$ mkdir build && echo "out" > build/result.txt
```

Step 2: See the mess

```
$ git status
Untracked files:
        app.log
        build/
        notes.txt
        secret.txt
```

Step 3: Write the do-not-pack list

```
$ printf "secret.txt\n*.log\nbuild/\n" > .gitignore
$ git status
Untracked files:
        .gitignore
        notes.txt
```

Step 4: Commit it

```
$ git add .
$ git commit -m "Add notes and .gitignore"
[main (root-commit) 4d2e7b1] Add notes and .gitignore
 2 files changed, 4 insertions(+)
```

Step 5: Peek at what Git is skipping

```
$ git status --ignored
Ignored files:
        app.log
        build/
        secret.txt
```

Step 6: Ask which rule is hiding a file

```
$ git check-ignore -v app.log
.gitignore:2:*.log    app.log
```

4. The gotcha: files Git already tracks

`.gitignore` only affects files Git is **not tracking yet**. If you committed a file first and ignored it later, Git keeps tracking it. Fix it like this:

```
$ git rm --cached scratch.txt
rm 'scratch.txt'
$ git commit -m "Stop tracking scratch.txt"
```

`--cached` removes the file from Git but **keeps it on your disk**.

> **Warning**
> If a password was ever committed, ignoring it later is not enough: it still lives in your history. Change the password.

5. Cheat sheet

Command	What it does
echo "*.log" >> .gitignore	Adds a rule
git status --ignored	Shows ignored files too
git check-ignore -v <file>	Which rule ignores this file?
git rm --cached <file>	Stop tracking a file, keep it on disk
git add .gitignore	Share the rules with your team

Ready-made lists for every language: `github.com/github/gitignore`

Your Day 8 commit
File: `days-1-to-8-summary.md`
Message: `docs(day08): add days 1-8 summary and gitignore notes`

6. Mermaid diagram GitHub draws for you

```
```mermaid
flowchart LR
  F["New file"] --> Q{"Matches .gitignore?"}
  Q -- Yes --> I["Ignored: Git never sees it"]
  Q -- No --> U["Untracked: can be added"]
```
```

7. Challenge and quiz

Challenge: create `.DS_Store` and `notes.tmp`. Write two rules so Git ignores both, then prove it with `git status --ignored`.

Q1. What does *.log match?

Q2. You ignored a file but Git still tracks it. How do you fix that?

Q3. Which kind of file should you never commit?

Answers: 1) Every file ending in .log. 2) git rm --cached <file>, then commit. 3) Passwords, keys and .env secrets.

Coming up: Day 9
git restore and git reset. Oops! Learn to take back mistakes safely.